Aprendizagem de Máquina

Aula 05 — Regressão Linear e Não Linear

Prof. Dr. Jonas Krause

PUCPR · PPGCIS · Ciência da Computação

03 de setembro de 2026

O plano de ensino inicia os modelos supervisionados

Semanas 5–7

Regressão e classificação. Implementação de modelos supervisionados.

Aula 05

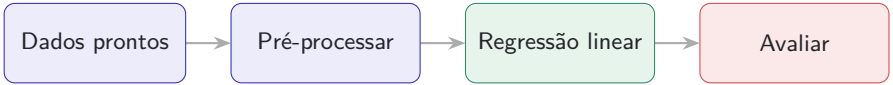
Regressão linear simples, múltipla e primeira extensão não linear.

- target contínuo;
- regressão linear simples e múltipla;
- regressão polinomial;
- métricas e resíduos.

Continuação

- Aula 06: classificação;
- Aula 07: consolidação dos modelos supervisionados;
- modelos mais complexos: etapas posteriores do plano.

A Aula 04 preparou os dados; agora treinamos o primeiro modelo



Continuidade

O mesmo pipeline sem vazamento da Aula 04 será usado. A novidade é o estimador que aprende a relação entre as features e um target contínuo.

Ao final, a equipe deverá comparar relações lineares e não lineares

1. reconhecer um problema de regressão;
2. diferenciar target contínuo de classe numérica;
3. compreender regressão linear simples e múltipla;
4. reconhecer quando uma reta não representa bem os dados;
5. construir uma regressão polinomial de grau 2;
6. treinar `LinearRegression` dentro de um pipeline;
7. calcular e interpretar MAE, MSE, RMSE e R^2;
8. analisar gráficos de valores previstos e resíduos;
9. comparar o modelo com um baseline simples;
10. comparar baseline, regressão linear e regressão polinomial;
11. aplicar o fluxo ao dataset pronto da equipe.

A aula avança do problema para a avaliação

Parte I — Construir

1. identificar um problema regressivo;
2. compreender a equação linear;
3. ajustar regressões simples e múltiplas;
4. reconhecer relações não lineares.

Parte II — Avaliar

5. construir uma regressão polinomial;
6. medir e visualizar os erros;
7. comparar os três fluxos;
8. interpretar limitações.

Exercício 1

Baseline e regressão linear em um primeiro diagnóstico.

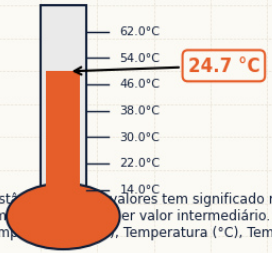
Exercício 2

Aplicação completa no dataset da equipe.

REGRESSÃO VS CLASSIFICAÇÃO: O FORMATO DO TARGET

REGRESSÃO: O TARGET CONTÍNUO

O Alvo representa uma quantidade numérica em uma escala contínua



- A distância entre os valores tem significado real.
- Permite qualquer valor intermediário.
- Exemplos: Preço ($), Temperatura (°C), Tempo (min).

CLASSIFICAÇÃO: O TARGET DISCRETO

O Alvo representa categorias rígidas, sem valores intermediários.



- Os números servem apenas como índices.
- Não existe transição entre elementos.
- Exemplos: Spam/Não-Spam, Fraude (0/1), Tipo de Peça.

DIRETRIZ DA LINHA DE MONTAGEM

O que define a tarefa é a natureza física do Target (y). Use Regressão para quantidades métricas contínuas e Classificação para portas de decisão rígi

Regressão prevê uma quantidade

Definição

Regressão é uma tarefa supervisionada em que o target representa uma quantidade numérica em uma escala.

Exemplos

- preço de um imóvel;
- consumo de energia;
- temperatura;
- tempo de entrega;
- produtividade agrícola.

Atenção

Um código numérico pode representar uma categoria. Prever classes 0, 1 e 2 continua sendo classificação.

O significado do target define a tarefa

Pergunta	Tarefa	Target
Qual será o consumo mensal?	Regressão	kWh
O cliente irá cancelar?	Classificação	sim / não
Quanto tempo durará a viagem?	Regressão	minutos
Qual espécie aparece na imagem?	Classificação	categoria
Qual será a temperatura interna?	Regressão	graus Celsius

Pergunta prática

A distância entre os valores do target possui significado quantitativo? Se sim, provavelmente temos regressão.

Uma boa regressão começa com uma previsão bem formulada

1. **Unidade de análise**: o que representa cada linha?
2. **Target**: qual quantidade será prevista?
3. **Unidade**: reais, kWh, °C, minutos ou outra?
4. **Momento da previsão**: quando o modelo será usado?
5. **Features disponíveis**: quais informações existem nesse momento?
6. **Erro aceitável**: quanto podemos errar antes de a previsão perder utilidade?

O contexto vem antes da métrica

Um erro de 5 pode ser pequeno para preço em milhares de reais e muito grande para temperatura em graus Celsius.

Nesta aula, o dataset já está pronto para modelagem

Ponto de partida

- dataframe carregado;
- target definido;
- ausentes, duplicatas e inconsistências tratados;
- features numéricas e categóricas identificadas;
- estratégia de divisão definida;
- pré-processamento encapsulado no pipeline.

Foco

Vamos treinar o primeiro modelo supervisionado e aprender a interpretar suas previsões.

O baseline mostra se as features realmente ajudam

Baseline de regressão

Uma referência simples que prevê sempre a média do target observada no conjunto de treino.

Se a regressão vence

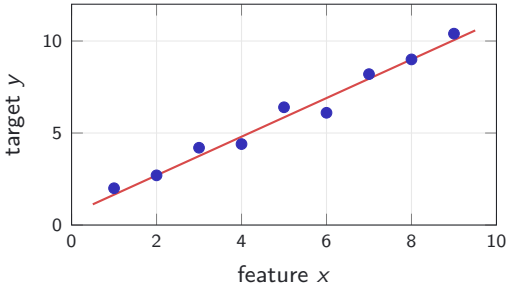
As features carregam algum sinal útil para prever o target.

Se não vence

Antes de buscar outro modelo, revise problema, target, features, qualidade dos dados e divisão.

```
DummyRegressor(strategy="mean")
```

A regressão linear procura uma tendência



Objetivo

Encontrar uma linha que represente a tendência dos dados e produza erros pequenos.

Importante

A linha não precisa passar por todos os pontos.

Uma feature produz uma regressão linear simples

Modelo

$$\widehat{y} = \beta_0 + \beta_1 x$$

β_0 — intercepto

Previsão quando $x = 0$. Sua interpretação depende de zero ser plausível para a feature.

β_1 — inclinação

Mudança esperada em $\widehat{y}$ quando x aumenta uma unidade.

O coeficiente transforma a feature em previsão

Considere:

$$\widehat{\text{preço}} = 80\,000 + 3\,000 \times \text{área}$$

- intercepto: R$ 80.000 quando a área é zero;
- inclinação: cada unidade adicional de área acrescenta R$ 3.000 à previsão;
- para área igual a 50:

$$\widehat{\text{preço}} = 80\,000 + 3\,000(50) = 230\,000$$

Cuidado

O coeficiente descreve uma associação aprendida nos dados. Ele não demonstra causalidade.

Várias features produzem uma regressão linear múltipla

Modelo

$$\widehat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

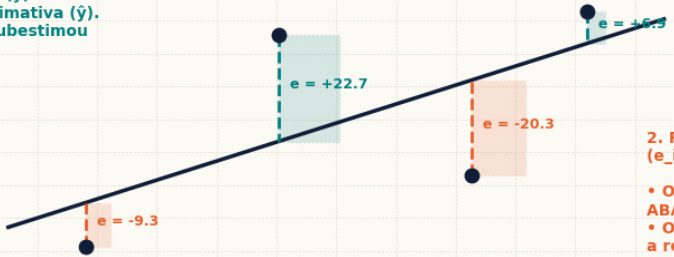
- p é o número de features;
- o modelo estima $p + 1$ coeficientes: $\beta_0, \beta_1, \ldots, \beta_p$;
- cada coeficiente está associado a uma feature;
- a previsão combina todas as contribuições;
- cada coeficiente é interpretado mantendo as demais features constantes.

O SENSOR DE DESVIOS: O QUE É O RESÍDUO?

1. RESÍDUO POSITIVO (e_i > 0)

- O valor real (y) ficou ACIMA da estimativa (ŷ).
- O modelo subestimou a realidade.

Resíduo: e_i = y_i - y_pred_i



2. RESÍDUO NEGATIVO (e_i < 0)

- O valor real (y) ficou ABAIXO da estimativa (ŷ).
- O modelo superestimou a realidade.

A MECÂNICA DO OLS (ORDINARY LEAST SQUARES)

O algoritmo OLS ajusta os coeficientes rotacionando a reta de forma a MINIMIZAR a soma das ÁREAS de todos os quadrados formados pelos erros. Em

O resíduo mede o erro de cada observação

Para a observação i

$$e_i = y_i - \widehat{y}_i$$

Resíduo positivo

O valor real ficou acima da previsão: o modelo subestimou.

Resíduo negativo

O valor real ficou abaixo da previsão: o modelo superestimou.

Exemplo

Valor real $= 250$; previsão $= 230$; resíduo $= 250 - 230 = 20$.

OLS escolhe os coeficientes que reduzem os erros quadráticos

Ordinary Least Squares

$$\min_{\beta_0,\ldots,\beta_p} \sum_{i=1}^{n} (y_i - \widehat{y}_i)^2$$

Índice $i = 1, \ldots, n$

n é o número de observações, isto é, as linhas usadas no ajuste.

Coeficientes $\beta_0, \ldots, \beta_p$

p é o número de features. Portanto, n não precisa ser igual a p.

Consequência

Erros grandes recebem peso quadrático e influenciam fortemente o ajuste.

O modelo entra no pipeline da Aula 04

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

pre = ColumnTransformer([
    ("num", Pipeline([
        ("imp", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ]), num_cols),
    ("cat", Pipeline([
        ("imp", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), cat_cols)
])

model = Pipeline([("pre", pre), ("reg", LinearRegression())])
```

O treino ajusta; o teste apenas recebe previsões

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Em `fit`

O pré-processamento aprende parâmetros no treino e a regressão aprende os coeficientes.

Em `predict`

O teste é apenas transformado e recebe as previsões. Nenhum parâmetro é reaprendido.

A BALANÇA DAS MÉTRICAS: MAE VS RMSE

1. MAE (ERRO ABSOLUTO MÉDIO)

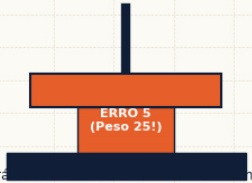
A Balança Comercial de Peso Linear



- Peso linear: cada unidade de erro custa exatamente o mesmo.
- Interpretação direta: erro médio na mesma unidade do target.
- Excelente para relatórios de impacto financeiro direto.

2. RMSE (RAIZ DO ERRO QUADRÁTICO)

A Prensa de Amplificação Quadrática



- Peso quadrá nte amplificados.
- Retorna à unidade original do target após extrair a raiz.
- Ideal para prever quebras críticas ou desvios intoleráveis.

DIAGNOSTICO DE MÉTRICAS

Regra de ouro de inspeção: se o seu RMSE estiver muito acima do seu MAE, significa que o seu modelo está cometendo alguns poucos erros gigante

Uma única métrica não descreve todos os erros

Duas perguntas diferentes:

1. Qual é o tamanho médio dos erros?
2. Existem alguns erros muito grandes?

Métricas complementares

- MAE: erro absoluto médio;
- MSE: erro quadrático médio;
- RMSE: raiz do erro quadrático médio;
- R^2: ganho em relação à previsão pela média.

A métrica precisa da unidade do problema

Um número isolado não informa se o desempenho é aceitável.

MAE responde qual é o erro absoluto médio

Definição

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \widehat{y}_i|$$

- mantém a mesma unidade do target;
- atribui peso linear a cada erro;
- possui interpretação direta;
- o melhor valor possível é zero.

Leitura

MAE = 2,4 °C significa que o erro absoluto médio foi de 2,4 °C.

MSE e RMSE destacam os erros maiores

MSE

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2$$

Como os erros são elevados ao quadrado, a unidade também fica ao quadrado.

RMSE

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

Retorna à unidade do target e mantém maior sensibilidade a erros elevados.

Diagnóstico

Se RMSE estiver muito acima de MAE, investigue observações com erros grandes.

R^2 compara o modelo com a previsão pela média

Coeficiente de determinação

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Valor	Interpretação
1	previsões perfeitas no conjunto avaliado;
0	desempenho equivalente a prever sempre a média;
< 0	desempenho pior que a referência da média.

Cuidado

R^2 alto não garante que o erro seja pequeno na unidade real do problema.

A métrica principal deve refletir o custo do erro

Métrica	Priorize quando	Pergunta respondida
MAE	cada unidade de erro tem custo semelhante	Qual é o erro típico?
RMSE	erros grandes são especialmente prejudiciais	Existem falhas graves?
R^2	deseja medir ganho sobre a média	Quanto da variação foi capturado?

Boa prática

Defina uma métrica principal e reporte pelo menos uma métrica complementar.

As métricas são calculadas sobre dados não vistos

```python
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    root_mean_squared_error,
    r2_score,
)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MAE : {mae:.3f}")
print(f"MSE : {mse:.3f}")
print(f"RMSE: {rmse:.3f}")
print(f"R2 : {r2:.3f}")
```

Exercício 1 — a regressão linear vence o baseline?

Objetivo

Verificar se as features do dataset carregam sinal útil para prever um target contínuo.

1. selecione o target contínuo;
2. declare sua unidade;
3. preserve o split e o pipeline;
4. treine o baseline pela média;
5. treine a regressão linear;
6. compare MAE, RMSE e R^2.

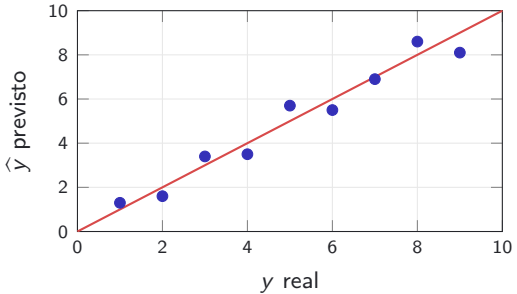
Tempo

20 minutos
Trabalho em equipe.

Produto

Tabela com os dois modelos e três conclusões.

Valores próximos da diagonal indicam previsões melhores



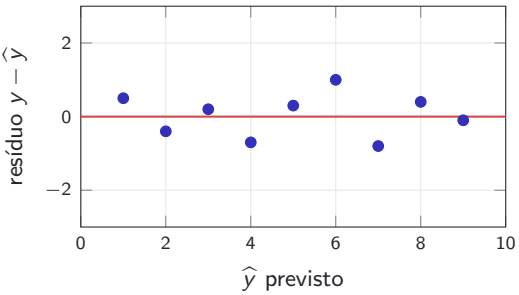
Leitura

Pontos próximos da diagonal indicam previsões próximas dos valores reais.

Procure

Faixas sistematicamente acima ou abaixo, dispersão crescente e pontos isolados.

Os resíduos revelam padrões escondidos pela média



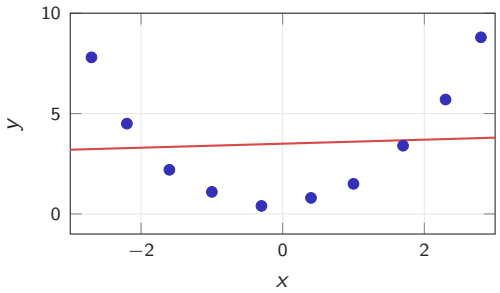
Padrão desejável

Nuvem sem estrutura clara e aproximadamente centrada em zero.

Sinais de problema

Curva, funil, grupos separados ou poucos resíduos extremos.

Uma relação curva deixa estrutura que a reta não consegue explicar



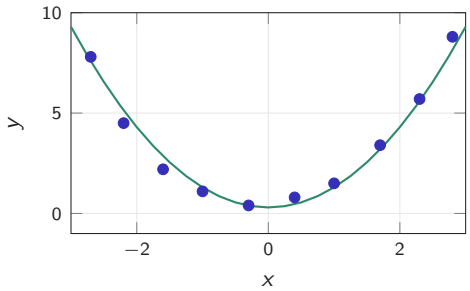
Problema

Uma reta resume apenas crescimento ou redução com inclinação constante.

Próximo passo

Criar features que representem curvatura, como x^2, e ajustar novamente.

A regressão polinomial representa uma relação não linear



Grau 2

$$\widehat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

Permite uma curva com uma mudança de direção.

Grau elevado

Aumenta a flexibilidade, mas também o risco de ajustar ruído e produzir previsões instáveis.

A curva continua linear em relação aos coeficientes

Transformação das features

$$x \quad \longrightarrow \quad [x,\ x^2]$$

- a não linearidade aparece na relação entre x e $\widehat{y}$;
- os coeficientes $\beta_0, \beta_1, \beta_2$ continuam combinados linearmente;
- `PolynomialFeatures` cria as novas colunas;
- `LinearRegression` estima os coeficientes;
- o grau do polinômio deve ser definido antes da avaliação final.

Nesta aula

Usaremos grau 2 para compreender o princípio sem transformar a atividade em busca de hiperparâmetros.

PolynomialFeatures e LinearRegression formam o modelo não linear

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

polynomial_model = Pipeline([
    ("polynomial", PolynomialFeatures(
        degree=2,
        include_bias=False
    )),
    ("regression", LinearRegression())
])

X_train_num = X_train[selected_num_cols]
X_test_num = X_test[selected_num_cols]
polynomial_model.fit(X_train_num, y_train)
pred_polynomial = polynomial_model.predict(X_test_num)
```

Escopo

A expansão polinomial deve ser aplicada a features numéricas selecionadas. Muitas features geram rapidamente muitas combinações.

Simples, múltipla e polinomial respondem a necessidades diferentes

Regressão	Entradas	Relação representada
Linear simples	uma feature	reta em uma dimensão
Linear múltipla	várias features	hiperplano com efeitos combinados
Polinomial	x, x^2, interações	curva ou superfície não linear

Critério de comparação

Use o mesmo treino, o mesmo teste e as mesmas métricas. A maior flexibilidade só é útil quando reduz o erro fora do treino.

Treino e teste ajudam a distinguir subajuste de sobreajuste

Situação	Sinal	Interpretação
Subajuste	erro alto no treino e no teste	a relação linear não captura estrutura suficiente
Bom resultado	erros baixos e próximos	o padrão aprendido generaliza
Sobreajuste	treino muito melhor que teste	o fluxo se adaptou demais ao treino

Nesta aula

A regressão linear é nosso primeiro modelo. Se ela falhar, o diagnóstico orientará as próximas etapas do plano.

Compare o erro do treino com o erro do teste

```
pred_train = model.predict(X_train)
pred_test = model.predict(X_test)

mae_train = mean_absolute_error(y_train, pred_train)
mae_test = mean_absolute_error(y_test, pred_test)

rmse_train = root_mean_squared_error(y_train, pred_train)
rmse_test = root_mean_squared_error(y_test, pred_test)

print("MAE treino:", mae_train)
print("MAE teste :", mae_test)
print("RMSE treino:", rmse_train)
print("RMSE teste :", rmse_test)
```

Leitura

Uma diferença grande exige investigação. Não significa automaticamente que o problema está resolvido apenas trocando o modelo.

Antes de concluir, descubra onde o modelo erra

- erros aumentam para valores altos do target?
- existe um período, região ou grupo com pior desempenho?
- poucos casos concentram grande parte do RMSE?
- as features usadas existem no momento real da previsão?
- o teste representa o cenário de uso?

Teste final

Consultar o teste repetidamente para alterar decisões transforma o teste em parte do desenvolvimento.

Conclusão responsável

Relate desempenho global, padrões de erro, limitações e situações em que o modelo não deve ser usado.

Os três fluxos devem receber as mesmas features numéricas

```
models = {
    "Baseline": DummyRegressor(strategy="mean"),
    "LinearMultipla": LinearRegression(),
    "PolinomialGrau2": Pipeline([
        ("poly", PolynomialFeatures(
            degree=2,
            include_bias=False
        )),
        ("reg", LinearRegression())
    ])
}

predictions = {}
for name, model in models.items():
    model.fit(X_train_num, y_train)
    predictions[name] = model.predict(X_test_num)
```

Comparação justa

Use de uma a três features numéricas, o mesmo treino e o mesmo teste nos três fluxos.

Uma tabela permite comparar ganho e custo da flexibilidade

```
import pandas as pd

def regression_metrics(y_true, y_pred):
    return {
        "MAE": mean_absolute_error(y_true, y_pred),
        "MSE": mean_squared_error(y_true, y_pred),
        "RMSE": root_mean_squared_error(y_true, y_pred),
        "R2": r2_score(y_true, y_pred),
    }

rows = []
for name, pred in predictions.items():
    rows.append({
        "Modelo": name,
        **regression_metrics(y_test, pred)
    })

table = pd.DataFrame(rows).sort_values("RMSE")
print(table.round(3))
```

Interpretação

A regressão polinomial só representa avanço quando reduz o erro de teste sem produzir uma diferença excessiva em relação ao treino.

O melhor resultado numérico ainda precisa de diagnóstico visual

```
import matplotlib.pyplot as plt

best_pred = predictions["PolinomialGrau2"] # adapte apos comparar
residuals = y_test - best_pred
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].scatter(y_test, best_pred, alpha=0.7)
axes[0].plot([y_test.min(), y_test.max()],
             [y_test.min(), y_test.max()], "r--")
axes[0].set(xlabel="Real", ylabel="Previsto")

axes[1].scatter(best_pred, residuals, alpha=0.7)
axes[1].axhline(0, color="red", linestyle="--")
axes[1].set(xlabel="Previsto", ylabel="Residuo")

plt.tight_layout()
plt.show()
```

A forma dos resíduos ajuda a escolher entre reta e curva

Antes de concluir, responda:

1. a regressão linear múltipla venceu o baseline?
2. a regressão polinomial melhorou o teste?
3. qual é o erro na unidade real do target?
4. RMSE e MAE estão próximos ou distantes?
5. os resíduos parecem aleatórios?
6. existe alguma faixa em que o modelo erra mais?

Não escolher apenas pela menor métrica

Uma pequena melhoria pode não justificar maior complexidade. Prefira a regressão linear quando os resultados forem equivalentes.

Exercício 2 — atividade final com o dataset da equipe

1. escolha um target contínuo e até três features numéricas;
2. preserve 20% dos dados para teste;
3. treine baseline, regressão linear múltipla e polinomial de grau 2;
4. compare MAE, MSE, RMSE e R^2;
5. compare erros de treino e teste;
6. gere real $\times$ previsto e resíduos $\times$ previsto;
7. escolha o fluxo e justifique em 5–8 linhas.

Tempo

35–40 minutos
Trabalho em equipe.

Regra

Não altere o grau após observar repetidamente o teste.

Produto

Notebook reproduzível + tabela de métricas + dois gráficos + conclusão.

O notebook final deve documentar todas as decisões

- ☐ problema, target e unidade;
- ☐ features escolhidas;
- ☐ split treino/teste;
- ☐ baseline pela média;
- ☐ regressão linear múltipla;
- ☐ regressão polinomial;
- ☐ tabela de métricas;
- ☐ gráfico real $\times$ previsto;
- ☐ gráfico de resíduos;
- ☐ conclusão de 5–8 linhas.

Critério de qualidade

Outra pessoa deve conseguir reproduzir o experimento e compreender por que a reta ou a curva foi escolhida.

A Aula 05 estabelece a base da aprendizagem supervisionada

Hoje consolidamos:

- regressão para targets contínuos;
- baseline como referência mínima;
- regressão linear simples e múltipla;
- regressão polinomial de grau 2;
- diferença entre relação linear e não linear;
- diferença entre n observações e p features;
- ajuste por mínimos quadrados;
- MAE, MSE, RMSE e R^2;
- gráficos de valores previstos e resíduos;
- diagnóstico de subajuste e sobreajuste;
- comparação entre baseline, reta e curva;
- aplicação completa no dataset da equipe.

Próxima aula — Classificação

Classes, probabilidades, matriz de confusão, precisão, recall e F1.

Referências

- GÉRON, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. 3. ed. O'Reilly, 2022.
- JAMES, G.; WITTEN, D.; HASTIE, T.; TIBSHIRANI, R. *An Introduction to Statistical Learning: with Applications in Python*. 2. ed. Springer, 2023.
- HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning*. 2. ed. Springer, 2009.
- SCIKIT-LEARN. *Linear Models*. `https://scikit-learn.org/stable/modules/linear_model.html`
- SCIKIT-LEARN. *Metrics and scoring*. `https://scikit-learn.org/stable/modules/model_evaluation.html`

Perguntas?

Aprendizagem de Máquina — Aula 05

Prof. Dr. Jonas Krause · PUCPR